

Ứng dụng của các cấu trúc dữ liệu cơ bản trong thi tin học

Zhu Chenguang
Trường THPT số 1 Vu Hồ, An Huy

Trại đông Tin học toàn quốc, 2006

Nguồn gốc: Ứng dụng của các cấu trúc dữ liệu cơ bản trong thi tin học

IOI China candidate team papers / 2006 / Zhu Chenguang.doc

Tóm tắt

Bài viết giới thiệu một số cấu trúc dữ liệu cơ bản, như danh sách tuyến tính, ngăn xếp và hàng đợi, cùng cách chúng được dùng trong các bài toán thi tin học. Thông qua ba ví dụ, tác giả nhấn mạnh rằng những cấu trúc tưởng như đơn giản có thể thay thế các cấu trúc phức tạp hơn, giảm độ khó cài đặt mà vẫn giữ được độ phức tạp tốt. Cuối bài viết là phần nhìn lại quan hệ giữa cấu trúc dữ liệu cơ bản và cấu trúc dữ liệu nâng cao.

Từ khóa: cấu trúc dữ liệu cơ bản; danh sách tuyến tính; hàng đợi; danh sách liên kết đôi; ngăn xếp; độ phức tạp cài đặt; độ phức tạp thời gian; quan hệ biện chứng.

1 Dẫn nhập

Trong các kỳ thi tin học ngày nay, bài khó xuất hiện ngày càng nhiều. Đi cùng với chúng thường là độ phức tạp cài đặt rất cao. Khoa học máy tính phát triển đã sinh ra nhiều cấu trúc dữ liệu hiệu quả và thực dụng, nhưng độ phức tạp cài đặt của chúng khiến thí sinh phải hết sức thận trọng; chỉ một sơ suất nhỏ cũng có thể làm hỏng toàn bộ chương trình.

Tuy nhiên, không phải bài nào cũng cần cấu trúc dữ liệu phức tạp. Nhiều lúc, những cấu trúc cơ bản mà ta dễ bỏ qua lại có đất dụng võ. Vận dụng linh hoạt các cấu trúc ấy có thể tiết kiệm thời gian quý trong phòng thi và tăng xác suất viết đúng chương trình.

2 Một số cấu trúc dữ liệu cơ bản

Phần này chỉ nhắc lại ngắn gọn các khái niệm vốn có trong mọi giáo trình cấu trúc dữ liệu.

2.1 Danh sách tuyến tính

Danh sách tuyến tính là một trong các cấu trúc thường gặp và đơn giản nhất. Nói ngắn gọn, một danh sách tuyến tính là một dãy hữu hạn gồm n phần tử dữ liệu. Các phép toán cơ bản trên danh sách tuyến tính gồm:

1. INITIATE(L): khởi tạo;
2. LENGTH(L): lấy độ dài;

3. GET(L, i): lấy phần tử thứ i ;
4. PRIOR($L, element$): lấy phần tử đứng trước;
5. NEXT($L, element$): lấy phần tử đứng sau;
6. LOCATE(L, x): định vị phần tử;
7. INSERT(L, i, b): chèn trước vị trí i ;
8. DELETE(L, i): xóa vị trí i ;
9. EMPTY(L): kiểm tra rỗng;
10. CLEAR(L): xóa toàn bộ danh sách.

Lưu trữ tuần tự. Trong máy tính, cách biểu diễn đơn giản và phổ biến nhất là dùng một dãy ô nhớ có địa chỉ liên tiếp để lưu lần lượt các phần tử; nói cách khác, đó là mảng một chiều. Nếu phần tử đầu ở địa chỉ b , mỗi phần tử chiếm L đơn vị bộ nhớ, thì phần tử thứ i ở địa chỉ $b + (i - 1)L$. Cách lưu trữ này cho phép truy cập ngẫu nhiên: lấy phần tử bất kỳ trong $O(1)$. Trong Pascal hoặc C, ta mô tả nó bằng mảng một chiều.

Trong cấu trúc lưu trữ tuần tự, chèn hoặc xóa một phần tử bất kỳ tốn $O(n)$, vì có thể phải dời nhiều phần tử; định vị theo chỉ số tốn $O(1)$.

Lưu trữ liên kết. Với danh sách liên kết đơn, mỗi nút lưu dữ liệu và một con trỏ tới nút kế tiếp. Nút đầu giả head giữ con trỏ tới phần tử đầu tiên. Các nút không cần nằm liên tiếp trong bộ nhớ.

Danh sách liên kết vòng là biến thể trong đó con trỏ của nút cuối trở lại nút đầu giả, tạo thành một vòng. Danh sách liên kết đôi thì mỗi nút có hai con trỏ: một tới nút kế tiếp và một tới nút đứng trước. Nó cũng có thể được tổ chức thành dạng vòng.

Trong cấu trúc lưu trữ liên kết, nếu vị trí đã được định vị thì chèn hoặc xóa tốn $O(1)$. Ngược lại, tìm vị trí của một phần tử thường tốn $O(n)$.

2.2 Ngăn xếp

Ngăn xếp là danh sách tuyến tính chỉ cho phép chèn và xóa ở cuối danh sách. Đầu cuối ấy gọi là đỉnh ngăn xếp; đầu còn lại gọi là đáy. Ngăn xếp rỗng là ngăn xếp không có phần tử.

Các thao tác sửa đổi của ngăn xếp tuân theo nguyên tắc vào sau ra trước (**last in, first out**, viết tắt là LIFO). Các phép toán cơ bản gồm:

- INISTACK(S): khởi tạo;
- EMPTY(S): kiểm tra rỗng;
- PUSH(S, x): đưa x vào ngăn xếp;
- POP(S, x): lấy phần tử ở đỉnh ra;
- GETTOP(S): xem phần tử ở đỉnh;
- CLEAR(S): xóa toàn bộ;
- CURRENT_SIZE(S): lấy số phần tử hiện có.

Mỗi thao tác cơ bản trên ngăn xếp đều tốn $O(1)$.

2.3 Hàng đợi

Ngược với ngăn xếp, hàng đợi là danh sách tuyến tính vào trước ra trước (**first in, first out**, viết tắt là FIFO). Nó chỉ cho phép chèn ở một đầu và xóa ở đầu còn lại. Đầu được chèn gọi là cuối hàng; đầu được xóa gọi là đầu hàng.

Các phép toán cơ bản gồm:

- $INITQUEUE(Q)$: khởi tạo;
- $EMPTY(Q)$: kiểm tra rỗng;
- $ENQUEUE(Q, x)$: đưa x vào hàng;
- $DLQUEUE(Q)$: lấy phần tử ở đầu hàng ra;
- $GETHEAD(Q)$: xem phần tử ở đầu hàng;
- $CLEAR(Q)$: xóa toàn bộ;
- $CURRENT_SIZE(Q)$: lấy số phần tử hiện có.

Mỗi thao tác cơ bản trên hàng đợi đều tốn $\mathcal{O}(1)$.

Ngoài ngăn xếp và hàng đợi còn có hàng đợi hai đầu. Ở đó, mỗi đầu có thể chỉ nhập, chỉ xuất, hoặc vừa nhập vừa xuất. Hàng đợi thường có thể xem là một trường hợp đặc biệt của hàng đợi hai đầu. Các cấu trúc cơ bản khác như chuỗi và cách lưu cây có gốc cũng rất quan trọng, nhưng bài viết này tập trung vào ngăn xếp, hàng đợi và danh sách tuyến tính.

3 Ứng dụng của ngăn xếp

3.1 Hình chữ nhật toàn số không lớn nhất

Bài toán. Cho một ma trận 0-1 kích thước $m \times n$, hãy tìm diện tích hình chữ nhật lớn nhất chỉ gồm các ô bằng 0.

Phân tích. Cách trực tiếp là liệt kê góc trái trên và góc phải dưới, rồi kiểm tra từng ô bên trong. Độ phức tạp là $\mathcal{O}(m^3n^3)$. Dùng tổng tiền tố hai chiều có thể kiểm tra một hình chữ nhật trong $\mathcal{O}(1)$, giảm tổng thời gian xuống $\mathcal{O}(m^2n^2)$, nhưng vẫn chưa đủ tốt.

Trong luận văn đội tuyển quốc gia năm 2003, Wang Zhikun đã nghiên cứu bài này và đưa ra thuật toán $\mathcal{O}(mn)$ dựa trên quy hoạch động. Bài viết này dùng một tính chất then chốt trong cách nhìn đó: trong mọi hình chữ nhật toàn số không cực đại đều tồn tại một “đường treo” thẳng đứng. Đường treo bắt đầu từ một ô và kéo lên trên cho tới khi gặp số 1 hoặc biên ma trận; hình chữ nhật cực đại có thể thu được bằng cách mở rộng đường treo ấy sang trái và sang phải.

Gọi $h(x, y)$ là độ dài đường treo kết thúc tại ô (x, y) . Ta có:

$$h(x, y) = \begin{cases} 0, & \text{nếu ô } (x, y) \text{ là } 1, \\ 1, & \text{nếu ô } (x, y) \text{ là } 0 \text{ và } x = 1, \\ h(x - 1, y) + 1, & \text{nếu ô } (x, y) \text{ là } 0 \text{ và } x > 1. \end{cases}$$

Sau khi tính $h(x, y)$ cho một hàng x , ta cần biết mỗi đường treo có thể mở rộng sang trái và sang phải bao xa mà không gặp chướng ngại. Cách làm trong bài này là duyệt hàng ấy bằng một ngăn xếp đơn điệu.

Để tránh xử lý trường hợp cuối hàng, thêm một lính canh $h(x, n + 1) = -1$. Trước khi xử lý mỗi hàng, làm rỗng ngăn xếp. Mỗi phần tử trong ngăn xếp lưu hai giá trị: chiều cao v và cột trái nhất j mà một đường treo có chiều cao v có thể vươn tới.

Khi xét cột i , gọi phần tử đỉnh là (A, j) .

- Nếu ngăn xếp rỗng hoặc $h(x, i) > A$, đưa $(h(x, i), i)$ vào ngăn xếp.
- Nếu $h(x, i) = A$, không cần chèn thêm, vì đường treo mới tạo ra cùng một tập hình chữ nhật như phần tử đã có, còn cột j đã ghi vị trí trái nhất.
- Nếu $h(x, i) < A$, liên tục lấy ra các phần tử cao hơn. Với mỗi phần tử (B, j) bị lấy ra, nó đã gặp biên phải ở cột i , nên diện tích ứng viên là $B(i - j)$. Sau khi lấy ra xong, nếu đỉnh mới có chiều cao bằng $h(x, i)$ thì không cần chèn; ngược lại chèn $h(x, i)$ với cột trái nhất bằng cột j của phần tử cuối cùng vừa bị lấy ra.

Trong mô tả hình của bản gốc, các phần tử trong ngăn xếp được vẽ như các điểm trên hệ trục: trục hoành là cột trái nhất, trục tung là chiều cao. Khi một chiều cao nhỏ hơn xuất hiện, các điểm cao hơn ở bên phải bị xóa vì chúng không thể tiếp tục mở rộng qua cột hiện tại.

Giải mã.

```
maxarea = 0
for x = 1..m:
    compute h(x, i) for all columns i
    h(x, n + 1) = -1
    clear stack
    for i = 1..n + 1:
        if stack is empty or h(x, i) > top.height:
            push(height = h(x, i), left = i)
        else if h(x, i) == top.height:
            continue
        else:
            lastLeft = i
            while stack is not empty and h(x, i) < top.height:
                lastLeft = top.left
                maxarea = max(maxarea, top.height * (i - top.left))
                pop
            if stack is empty or h(x, i) > top.height:
                push(height = h(x, i), left = lastLeft)
```

Mỗi phần tử được đưa vào và lấy ra khỏi ngăn xếp nhiều nhất một lần trong mỗi hàng, nên tổng độ phức tạp là $\mathcal{O}(mn)$. Dùng mảng cuộn cho h thì bộ nhớ còn $\mathcal{O}(n)$. Điểm khác với ngăn xếp thường là các chiều cao trong ngăn xếp luôn tăng nghiêm ngặt từ đáy tới đỉnh, và mỗi phần tử còn mang theo vị trí trái nhất tương ứng.

4 Ứng dụng của danh sách tuyến tính

4.1 Thống kê doanh thu

Bài toán. Cho doanh thu $a_1, a_2, \dots, a_n$ của n ngày, với $1 \leq n \leq 32767$. Giá trị dao động nhỏ nhất của ngày i được định nghĩa là

$$\min_{1 \leq j < i} |a_j - a_i|.$$

Riêng ngày đầu tiên có giá trị dao động nhỏ nhất bằng a_1 . Hãy tính tổng các giá trị dao động nhỏ nhất của mọi ngày.

Phân tích. Vòng lặp hai tầng cho độ phức tạp $\mathcal{O}(n^2)$, không đủ nhanh. Nếu dùng cây cân bằng, khi chèn doanh thu của ngày hiện tại ta có thể lấy giá trị gần nhất trên đường tìm kiếm và đạt $\mathcal{O}(n \log n)$. Nhưng cây cân bằng có độ phức tạp cài đặt cao: quay trái, quay phải và các biến thể đều dễ sai trong điều kiện thi.

Ta có thể đạt cùng bậc thời gian bằng danh sách liên kết đôi. Trước hết sắp xếp n phần tử theo giá trị, đồng thời ghi vị trí b_i của phần tử a_i trong dãy sau sắp xếp. Trên dãy đã sắp xếp, dựng một danh sách liên kết đôi. Sau đó xử lý các phần tử theo thứ tự ngược từ a_n về a_1 .

Khi xử lý a_i , các phần tử $a_{i+1}, a_{i+2}, \dots, a_n$ đã bị xóa khỏi danh sách. Vì vậy, tiền nhiệm và hậu nhiệm của vị trí b_i trong danh sách hiện tại chính là hai giá trị lân cận của a_i trong tập $\{a_1, \dots, a_i\}$ sau khi sắp xếp. Dao động nhỏ nhất của ngày i chỉ có thể là khoảng cách tới một trong hai giá trị đó. Sau khi cộng đáp án, xóa a_i khỏi danh sách trong $\mathcal{O}(1)$.

Ví dụ $a_1 = 9, a_2 = 3, a_3 = 8$. Sau khi sắp xếp ta có 3, 8, 9. Với $a_3 = 8$, hai láng giềng là 3 và 9, nên dao động là 1. Xóa 8. Với $a_2 = 3$, chỉ còn hậu nhiệm 9, nên dao động là 6. Ngày đầu đóng góp 9. Tổng là 16.

Giải mã.

```
input n and a[1..n]
sort the indices by increasing a[index]
record pos[i] = position of a[i] in the sorted order
build a doubly linked list over the sorted indices

answer = a[1]
for i = n downto 2:
    best = infinity
    if previous[i] exists:
        best = min(best, abs(a[i] - a[previous[i]]))
    if next[i] exists:
        best = min(best, abs(a[i] - a[next[i]]))
    answer += best
    remove i from the doubly linked list
print answer
```

Sắp xếp tốn $\mathcal{O}(n \log n)$, còn dựng danh sách và xử lý đều tốn $\mathcal{O}(n)$. Vì vậy tổng độ phức tạp là $\mathcal{O}(n \log n)$, ngang với cách dùng cây cân bằng nhưng mã ngắn và ít rủi ro hơn.

5 Ứng dụng của hàng đợi

5.1 Điều valse lộng lẫy

Bài toán. Cho một ma trận $n \times m$, một số ô là chướng ngại. Một người bắt đầu ở một ô trống và thực hiện k lượt trượt. Ở lượt thứ t , hướng trượt đã biết trước là đông, nam, tây hoặc bắc, và người đó có thể đi liên tiếp tối đa c_t ô theo hướng ấy, cũng có thể đứng yên. Trong quá trình trượt không được chạm chướng ngại. Hãy tìm quãng đường trượt dài nhất. Dữ liệu trong bản gốc có $1 \leq n, m \leq 200$, $k \leq 200$, tổng thời gian không quá 40000.

Quy hoạch động trực tiếp. Đặt $f(t, x, y)$ là quãng đường dài nhất sau t lượt nếu kết thúc ở ô (x, y) . Với lượt trượt sang đông, ta có dạng chuyển:

$$\begin{aligned} f(t, x, y) = \max \{ & f(t-1, x, y), \\ & f(t-1, x, y-1) + 1, \\ & f(t-1, x, y-2) + 2, \dots, \\ & f(t-1, x, y') + y - y' \}, \end{aligned}$$

trong đó y' là cột trái nhất còn có thể đi tới mà không vượt quá c_t ô và không đi xuyên qua chướng ngại. Ba hướng còn lại tương tự.

Số trạng thái là $\mathcal{O}(knm)$, nhưng mỗi chuyển có thể phải xét $\mathcal{O}(\max\{n, m\})$ ô, nên độ phức tạp quá lớn.

Biến đổi công thức. Tạm bỏ qua chướng ngại. Với một hàng cố định và hướng sang đông, nếu $c_t = 2$, ta có:

$$\begin{aligned} f(t, x, 1) &= \max\{f(t-1, x, 1)\}, \\ f(t, x, 2) &= \max\{f(t-1, x, 1) + 1, f(t-1, x, 2)\}, \\ f(t, x, 3) &= \max\{f(t-1, x, 1) + 2, f(t-1, x, 2) + 1, f(t-1, x, 3)\}, \\ f(t, x, 4) &= \max\{f(t-1, x, 2) + 2, f(t-1, x, 3) + 1, f(t-1, x, 4)\}. \end{aligned}$$

Đặt

$$a_i = f(t-1, x, i) - i + 1.$$

Khi đó các biểu thức trên trở thành việc lấy giá trị lớn nhất của một đoạn liên tiếp trong dãy a , rồi cộng thêm một hằng số theo cột hiện tại. Nếu có chướng ngại, các đoạn bị cắt rời; khi gặp chướng ngại ta phải xóa toàn bộ cửa sổ hiện tại.

Vì mỗi bước chỉ thêm một giá trị ở cuối cửa sổ và xóa một số giá trị ở đầu cửa sổ, cấu trúc phù hợp tự nhiên là hàng đợi. Nhưng ta cần lấy giá trị lớn nhất trong hàng, nên dùng **hàng đợi đơn điệu**.

Hàng đợi đơn điệu. Trong cửa sổ hiện tại, giả sử hai giá trị a_2 và a_3 cùng tồn tại và $a_2 \leq a_3$. Khi a_2 chưa bị xóa, a_3 chắc chắn cũng chưa bị xóa, vì a_3 được thêm sau. Do đó a_2 không bao giờ có thể là giá trị lớn nhất và không cần lưu. Từ nhận xét này suy ra các giá trị thật sự cần lưu trong hàng đợi phải giảm nghiêm ngặt từ đầu tới cuối; phần tử đầu hàng chính là giá trị lớn nhất của cửa sổ.

Khi xử lý một ô mới:

- nếu ô là chướng ngại, xóa rỗng hàng đợi;
- xóa khỏi đầu hàng mọi phần tử đã cách ô hiện tại quá c_t ;
- trước khi chèn a_i , xóa khỏi cuối hàng mọi phần tử không lớn hơn a_i ;

- chèn a_i , rồi dùng phần tử đầu hàng để cập nhật $f(t, x, y)$.

Hình minh họa của bản gốc vẽ các giá trị a_i trên hệ trục chỉ số–giá trị: các điểm thấp hơn nằm trước một điểm cao hơn ở bên phải bị loại bỏ, vì chúng không thể thắng trong bất kỳ cửa sổ tương lai nào.

Giải mã cho hướng đông.

```
for x = 1..n:
    clear deque
    for y = 1..m:
        if cell (x, y) is blocked:
            f[now][x][y] = -infinity
            clear deque
            continue
        while deque is not empty and deque.front.index < y - c[t]:
            pop_front
        value = f[old][x][y] - y + 1
        while deque is not empty and deque.back.value <= value:
            pop_back
        push_back(index = y, value = value)
        f[now][x][y] = deque.front.value + y - 1
```

Ba hướng còn lại chỉ khác thứ tự duyệt và cách tính khoảng cách. Mỗi ô vào và ra hàng đợi nhiều nhất một lần trong mỗi lượt, vì vậy tổng độ phức tạp là $\mathcal{O}(knm)$, tốt hơn cách dùng cây đoạn $\mathcal{O}(knm \log \max\{n, m\})$ và cũng dễ cài đặt hơn.

6 Tổng kết

Ba ví dụ trên cho thấy các cấu trúc dữ liệu cơ bản không chỉ là kiến thức nhập môn. Ngăn xếp, danh sách liên kết đôi và hàng đợi có thể, khi được tổ chức đúng cách, giải những bài tưởng như cần cấu trúc dữ liệu cao cấp hơn.

Điểm mấu chốt không phải là phủ nhận cấu trúc dữ liệu nâng cao. Cây cân bằng, cây đoạn và nhiều cấu trúc chuyên biệt vẫn cần thiết trong các tình huống phù hợp. Nhưng trong thi đấu, ta luôn phải cân bằng giữa độ phức tạp thời gian, độ phức tạp cài đặt và khả năng viết đúng. Cấu trúc cơ bản thường đem lại một phương án có độ phức tạp đủ tốt với mã ngắn và ít lỗi.

Tác giả cũng nhấn mạnh một khía cạnh tư tưởng: việc quay lại dùng cấu trúc cơ bản không phải là sự trở về đơn giản, càng không phải là loại bỏ tri thức nâng cao. Ta đi từ cấu trúc cơ bản tới cấu trúc nâng cao, rồi nhìn lại cấu trúc cơ bản ở một tầng hiểu biết cao hơn. Quá trình ấy giống một sự phát triển theo đường xoắn ốc: bề ngoài là trở về điểm cũ, nhưng thực chất nhận thức đã sâu hơn.

Tài liệu tham khảo

1. Yan Weimin, Wu Weimin. *Data Structures*, ấn bản thứ hai. Thanh Hoa University Press, 1992.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, Second Edition. The MIT Press, 2001.
3. Wang Zhikun. *Bản về cách dùng tư tưởng cực đại hóa để giải bài toán hình chữ nhật con lớn nhất*. Luận văn đội tuyển quốc gia IOI 2003.

Lời cảm ơn

Tác giả chân thành cảm ơn thầy Jiang Tao vì đã hướng dẫn và giúp đỡ trong quá trình viết bài; đồng thời cảm ơn huấn luyện viên Liu Rujia và bạn Xu Zhilei vì những góp ý quan trọng.

Phụ lục được lược dịch

Bản gốc kèm các phát biểu đầy đủ của ba bài ví dụ và ba chương trình tham khảo bằng C/C++. Theo yêu cầu của bản dịch này, phần chương trình dài không được chép lại. Nội dung phụ lục có thể khá quát như sau:

- Ví dụ 1 tương ứng bài *Big Barn* của USACO Training: tìm hình vuông lớn nhất không chứa cây trên lưới $N \times N$. Chương trình tham khảo dùng đúng ngăn xếp đơn điệu đã mô tả, chỉ thay diện tích hình chữ nhật bằng cạnh hình vuông lớn nhất.
- Ví dụ 2 là bài *Thống kê doanh thu*: đọc n doanh thu, tính tổng dao động nhỏ nhất. Chương trình tham khảo sắp xếp chỉ số rồi xóa ngược trên danh sách liên kết đôi.
- Ví dụ 3 là bài *Điều valse lộng lẫy*, mã nguồn chính adv1900: dùng quy hoạch động hai lớp và hàng đợi đơn điệu theo từng hàng hoặc cột cho mỗi khoảng thời gian nghiêng của con tàu.